

数据结构

Data Structures & Algorithms

MS Roslyn

2026 年 8 月 16 日

摘要

本教程系统介绍数据结构的核心概念与经典算法，涵盖位置记数系统、数据系统与物理存储、线性结构与抽象数据类型、树与二叉树、图论，以及算法定义与渐近分析、经典算法与核心运算，并通过不同算法的对比帮助读者理解时间与空间复杂度之间的权衡。教程将抽象数据类型与具体实现相结合，为程序设计提供坚实的数据组织基础。

目录

第一章	Positional Numeral System	5
1.1	任意进制定义	5
1.1.1	基数与合法数码集 (Alphabet of Digits)	5
1.1.2	离散数码序列 (Numeral Sequence)	5
1.1.3	按权展开映射 (Polynomial Expansion)	5
1.2	任意进制转换定义 ($n \rightarrow m$)	6
1.2.1	转换核心公理: 值守恒 (Value Conservation)	6
1.2.2	数学求解算子: 数码分离 (Digit Extraction Operator)	6
1.3	数学算子应用示例 ($10 \rightarrow 2$ 进制)	7
1.3.1	整数部分求解 ($y = 11$, 求解 $j \geq 0$ 的数码)	7
1.3.2	小数部分求解 ($y_f = 0.625$, 求解 $j < 0$ 的数码)	7
1.3.3	最终值守恒合并	8
1.4	算法级数轴递推 (计算机实现逻辑的状态转移方程)	8
1.4.1	整数项递推: 商与余数 (Division-Remainder Iteration)	8
1.4.2	小数项递推: 积与整数 (Multiplication-Integer Iteration)	8
第二章	Data System and Physical Storage	10
2.1	最根本元起点	10
2.2	数字体系 (Number)	10
2.2.1	自然数 ($\mathbb{N}$)	10
2.2.2	整数 ($\mathbb{Z}$)	10
2.2.3	实数/有理数 ($\mathbb{R} \rightarrow \mathbb{Q}_{\text{discrete}}$)	11
2.3	字符体系 (Character)	11
2.3.1	ANSI / ASCII	11
2.3.2	Unicode 架构	11
2.4	物理存储结构: 线性表 (Linear List)	11
2.4.1	顺序存储结构 (顺序表 / 数组)	12
2.4.2	链式存储结构 (链表 / Link List)	12
2.5	核心矛盾对比: 顺序表 vs 链表	13
第三章	Linear Structures and Abstract Data Types	14
3.1	栈 (Stack)	14
3.1.1	代数定义	14
3.1.2	物理实现方案	14

3.2	队列 (Queue)	15
3.2.1	代数定义	15
3.2.2	循环队列 (Circular Queue)	16
3.3	字符串 (String)	17
3.3.1	形式化定义	17
3.4	高级映射与哈希表体系	17
3.4.1	字典 (Dictionary) / 映射 (Map)	17
3.4.2	哈希表 (Hash Table)	17
3.4.3	哈希碰撞 (Hash Collision)	17
第四章	Non-linear Structures I: Trees and Binary Trees	20
4.1	通用树的基础属性	20
4.2	二叉树 (Binary Tree) 及其形态特征	20
4.3	树的物理存储	21
4.4	树的遍历 (Tree Traversal)	21
4.4.1	递归遍历 (Recursive Traversal)	22
4.4.2	非递归遍历 (Iterative Traversal)	22
4.4.3	线索二叉树 (Threaded Binary Tree)	22
4.4.4	遍历序列与树的唯一性	23
4.5	堆体系 (Heap)	23
第五章	Non-linear Structures II: Graph Theory	25
5.1	图的形式化定义	25
5.2	边的拓扑分类	25
5.3	连通性深度分析	25
5.4	图的降维与拓扑排序	26
5.5	图的物理存储实现	26
第六章	Algorithm Definition and Asymptotic Analysis	28
6.1	算法的定义与四大核心特征	28
6.2	算法的描述媒介	28
6.3	算法的时间复杂度 (Time Complexity)	29
6.3.1	大 O 表示法的渐进上界定义	29
6.3.2	复杂度阶数	29
6.3.3	复杂度组合乘法原理	29
6.4	算法的空间复杂度 (Space Complexity)	30
6.4.1	$O(1)$ 常数级辅助空间 (In-place 原地置换)	30
6.4.2	$O(n)$ 线性级辅助空间 (Clone 副本生成)	30
第七章	Dynamic Classic Algorithms and Core Operators	31
7.1	顶层设计范式	31
7.1.1	分治策略 (Divide and Conquer)	31
7.1.2	贪心算法 (Greedy)	31

7.2	排序算法算子 (Sort)	32
7.2.1	比较类排序	32
7.2.2	非比较类排序	32
7.3	查找与并查集	33
7.3.1	二分查找 (Binary Search)	33
7.3.2	并查集 (Union-Find)	33
7.4	图论核心算法	34
7.4.1	基础遍历	34
7.4.2	最短路径算法	34
7.4.3	最小生成树 (MST)	35
7.4.4	拓扑排序: Kahn 算法	35
7.5	高级工程专项算子	35
7.5.1	网络流 (Network Flow)	35
7.5.2	二分图最大匹配	35
7.5.3	强连通分量 (SCC): Tarjan 算法	35
第八章 Algorithm Comparison		36
8.1	核心排序算法算子对比	36
8.2	高级图论与检索算子对比	36

第一章 Positional Numeral System

在数学中， n 进制（进位计数制，Positional Numeral System）是指用一组固定的符号来表示任意实数的计数方法。其严谨的数学形式化定义由以下三个核心要素构成：

1.1 任意进制定义

1.1.1 基数与合法数码集 (Alphabet of Digits)

设基数 n 为一个大于 1 的正整数 ($n \in \mathbb{Z}^+, n > 1$)。该进制的有限数码集合 S_n 严格定义为：

$$S_n = \{x \mid x \in \mathbb{N}, 0 \leq x < n\} = \{0, 1, 2, \dots, n-1\}$$

1.1.2 离散数码序列 (Numeral Sequence)

任意实数在 n 进制下的形式化表达，是一个双向无穷的离散数码序列：

$$\{P_i\}_{i=-\infty}^{\infty}$$

其中，每一位上的数码必须满足严格约束： $\forall i \in \mathbb{Z}, P_i \in S_n$ 。

注：为保证映射的唯一性，通常规定最高有效位向左不包含无穷个连续的 0，且小数末尾不包含无穷个连续的数码 $(n-1)$ 。

1.1.3 按权展开映射 (Polynomial Expansion)

将”数码序列”映射到实数轴上的”绝对数值 y ” ($y \in \mathbb{R}$)，是通过位权级数求和实现的双向映射：

$$y = \sum_{i=-\infty}^{\infty} P_i \cdot n^i$$

将该无穷级数以小数点 ($i = 0$) 为分界展开，其标准数学形式为：

$$y = \dots + P_2 \cdot n^2 + P_1 \cdot n^1 + P_0 \cdot n^0 + P_{-1} \cdot n^{-1} + P_{-2} \cdot n^{-2} + \dots$$

- 整数部分 ($i \geq 0$)：位权为 n^i ，对应的基数为正指数幂。
- 小数部分 ($i < 0$)：位权为 n^i ，对应的基数为负指数幂。

1.2 任意进制转换定义 ($n \rightarrow m$)

将一个数从 n 进制转换到 m 进制，在数学上是通过值守恒公理与数码分离算子来进行形式化表达的。

1.2.1 转换核心公理：值守恒 (Value Conservation)

进制转换的本质是变换数码的形式，但保持数轴上的绝对值 y 不变。设源进制为 n (数码为 P_i)，目标进制为 m (数码为 Q_j)，则数学映射关系严格表达为：

$$y = \sum_{i=-\infty}^{\infty} P_i \cdot n^i = \sum_{j=-\infty}^{\infty} Q_j \cdot m^j$$

其中： $P_i \in S_n$, $Q_j \in S_m$ 。

1.2.2 数学求解算子：数码分离 (Digit Extraction Operator)

已知绝对值 y ，想要显式直接求解目标 m 进制中任意第 j 位上的数码 Q_j ，数学上使用向下取整算子 $\lfloor \dots \rfloor$ 和 取模算子 $\bmod$ 表达：

- 整数部分 (第 j 位数码，其中 $j \geq 0$):

$$Q_j = \left\lfloor \frac{y}{m^j} \right\rfloor \bmod m$$

(注：将值右移 j 位，截去低位干扰后取模)

- 小数部分 (第 j 位数码，其中 $j < 0$):

$$Q_j = \left\lfloor y_{\text{fraction}} \cdot m^{-j} \right\rfloor \bmod m$$

(注： $y_{\text{fraction}} = y - \lfloor y \rfloor$ 为纯小数部分。通过左移 $-j$ 位将目标小数位提到整数个位，取整后取模剥离)

1.3 数学算子应用示例 ($10 \rightarrow 2$ 进制)

【示例】将十进制数 $y = 11.625$ 转换为二进制 (目标基数 $m = 2$)。

1.3.1 整数部分求解 ($y = 11$, 求解 $j \geq 0$ 的数码)

- $j = 0$ (个位) :

$$Q_0 = \left\lfloor \frac{11}{2^0} \right\rfloor \bmod 2 = 11 \bmod 2 = 1$$

- $j = 1$ (十位/ 2^1 位) :

$$Q_1 = \left\lfloor \frac{11}{2^1} \right\rfloor \bmod 2 = \lfloor 5.5 \rfloor \bmod 2 = 5 \bmod 2 = 1$$

- $j = 2$ (百位/ 2^2 位) :

$$Q_2 = \left\lfloor \frac{11}{2^2} \right\rfloor \bmod 2 = \lfloor 2.75 \rfloor \bmod 2 = 2 \bmod 2 = 0$$

- $j = 3$ (千位/ 2^3 位) :

$$Q_3 = \left\lfloor \frac{11}{2^3} \right\rfloor \bmod 2 = \lfloor 1.375 \rfloor \bmod 2 = 1 \bmod 2 = 1$$

- $j = 4$: $\lfloor \frac{11}{2^4} \rfloor = 0$, 计算终止。得出整数部分: $(11)_{10} = (Q_3 Q_2 Q_1 Q_0)_2 = (1011)_2$

1.3.2 小数部分求解 ($y_f = 0.625$, 求解 $j < 0$ 的数码)

- $j = -1$ (十分位/ 2^{-1} 位) :

$$Q_{-1} = \lfloor 0.625 \cdot 2^{-(-1)} \rfloor \bmod 2 = \lfloor 1.25 \rfloor \bmod 2 = 1 \bmod 2 = 1$$

- $j = -2$ (百分位/ 2^{-2} 位) :

$$Q_{-2} = \lfloor 0.625 \cdot 2^{-(-2)} \rfloor \bmod 2 = \lfloor 2.5 \rfloor \bmod 2 = 2 \bmod 2 = 0$$

- $j = -3$ (千分位/ 2^{-3} 位) :

$$Q_{-3} = \lfloor 0.625 \cdot 2^{-(-3)} \rfloor \bmod 2 = \lfloor 5.0 \rfloor \bmod 2 = 5 \bmod 2 = 1$$

- $j = -4$: 小数部分已被完全提取, 后续结果均为 0, 计算终止。

得出小数部分: $(0.625)_{10} = (0.Q_{-1}Q_{-2}Q_{-3})_2 = (0.101)_2$

1.3.3 最终值守恒合并

$$\sum_{j=-3}^3 Q_j \cdot 2^j = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 11.625$$

确定最终转换结果：(11.625)₁₀ = (1011.101)₂

1.4 算法级数轴递推（计算机实现逻辑的状态转移方程）

在计算机实际运行中，如果直接对高维幂次 m^j 进行直接计算（如前面的直接算子），会带来极高的空间复杂度与浮点数精度误差。为了让硬件高效执行，必须将静态的数学公式转化为单步迭代的递推状态转移方程。

这便是计算机中经典的 ”商与余数递推（除基取余法）” 与 ”积与整数递推（乘基取整法）”。

1.4.1 整数项递推：商与余数 (Division-Remainder Iteration)

此算法用于处理**整数部分** ($j \geq 0$)。它通过引入中间状态变量 y_j ，在每一步循环中只做一次单步除法，从而把全局计算降维到局部计算。

(1) 状态转移方程组

$$\begin{cases} Q_j = y_j \bmod m & \text{(状态输出：当前步的数码)} \\ y_{j+1} = \lfloor \frac{y_j}{m} \rfloor & \text{(状态转移：更新下一步的被除数)} \end{cases}$$

- 初始状态： $y_0 = \lfloor y \rfloor$ （即原始数值的纯整数部分）。
- 终止条件： 当新状态 $y_{j+1} = 0$ 时，算法迭代停止。

(2) 核心原理解析

- $Q_j = y_j \bmod m$ ： 因为任意整数都可以写成 $y_j = k \cdot m + Q_j$ 的形式，所以对 m 取模可以直接隔离出当前最低位（末尾）的数码。
- $y_{j+1} = \lfloor \frac{y_j}{m} \rfloor$ ： 将当前数字整体向右移一位（抹去刚刚已经提取出的末尾数码），其商作为新的状态输入到下一轮循环中。
- 数码产出顺序： 由于我们是从个位开始向左剥离，因此数码的产出顺序是从**低位到高位**（即 $Q_0 \rightarrow Q_1 \rightarrow Q_2 \dots$ ）。计算机输出时需要**逆序**排列。

1.4.2 小数项递推：积与整数 (Multiplication-Integer Iteration)

此算法用于处理**小数部分** ($j < 0$)。由于小数点的右侧代表负指数幂 ($\frac{1}{m}, \frac{1}{m^2}$)，我们需要通过”放大”而非”缩小”来提取数码。

(1) 状态转移方程组

$$\begin{cases} Q_j = \lfloor y_j \cdot m \rfloor & \text{(状态输出: 当前步的数码)} \\ y_{j-1} = (y_j \cdot m) - Q_j & \text{(状态转移: 更新下一步的纯小数)} \end{cases}$$

- 初始状态: $y_{-1} = y - \lfloor y \rfloor$ (即原始数值的纯小数部分)。
- 终止条件: 新状态 $y_{j-1} = 0$ (即小数部分完全被榨干清零) 或达到指定的计算机最高精度步数。

(2) 核心原理解析

- $Q_j = \lfloor y_j \cdot m \rfloor$: 由于 y_j 是一个处于 $[0, 1)$ 区间内的纯小数, 将其乘以基数 m 后, 其原本在小数点后第一位的数字会被放大到整数部分 (个位), 通过向下取整即可直接摘取。
- $y_{j-1} = (y_j \cdot m) - Q_j$: 乘基之后, 减去刚刚脱离出来的整数部分 Q_j , 剩下的依然是一个纯小数。这个余下的纯小数作为新的状态, 继续投入下一次放大。
- 数码产出顺序: 因为我们是从十分位开始向右逐位放大剥离, 所以数码的产出顺序是从高位到低位 (即 $Q_{-1} \rightarrow Q_{-2} \rightarrow Q_{-3} \dots$)。计算机输出时按常规正序拼接即可。

第二章 Data System and Physical Storage

本章自底向上探讨数据在计算机底层的物理本质，从最基本的比特流开始，逐步演进到数字系统、字符系统以及物理内存的组织形式。

2.1 最根本元起点

- 基本域： $\mathbb{B} = \{0, 1\}$ 。
- 比特序列空间： $\mathbb{B}^* = \bigcup_{n=0}^{\infty} \mathbb{B}^n$ 。计算机中所有存储对象均可视为 $\mathbb{B}^*$ 中的元素。
- 解释协议 (Interpretation Protocols)：通过映射函数 $\mathcal{I} : \mathbb{B}^n \rightarrow \mathcal{D}$ ，将底层比特流解释为特定的数学对象集合 $\mathcal{D}$ （如整数、字符、地址等）。

2.2 数字体系 (Number)

2.2.1 自然数 ($\mathbb{N}$)

- 集合定义： $\mathbb{N} = \{0, 1, 2, \dots\}$ 。
- 有限位表示：若分配 n 位存储空间（即 $b \in \mathbb{B}^n$ ），其可表示的绝对数值范围为：

$$[0, 2^n - 1]$$

- 解码映射： $\mathcal{I}_{\mathbb{N}}(b) = \sum_{i=0}^{n-1} b_i \cdot 2^i$ 。

2.2.2 整数 ($\mathbb{Z}$)

- 集合定义： $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ 。
- 补码系统 (Two's Complement)：为统一加减法运算，引入补码映射。对于 n 位二进制序列 $b = b_{n-1}b_{n-2}\dots b_0 \in \mathbb{B}^n$ ，其解码方程为：

$$\mathcal{I}_{\mathbb{Z}}(b) = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

- 值域：该映射将 $\mathbb{B}^n$ 双射到 $[-2^{n-1}, 2^{n-1} - 1] \subset \mathbb{Z}$ ，消除了 $+0$ 与 -0 的二义性。

2.2.3 实数/有理数 ($\mathbb{R} \rightarrow \mathbb{Q}_{\text{discrete}}$)

物理事实：计算机为离散状态机，无法精确表达连续无理数。实数被离散化为有理数子集 $\mathbb{Q}_{\text{discrete}} \subset \mathbb{Q}$ 。

1. **定点数 (Fixed-Point)**: 小数点位置固定。数学表达为 $x = m \cdot B^{-f}$ ，其中 $m \in \mathbb{Z}$ ， f 为固定小数位数。
2. **浮点数 (Floating-Point)**:
 - **通用形式**: $x = (-1)^s \cdot m \cdot 2^e$ ，其中 $s \in \mathbb{B}$ ， m 为尾数， e 为指数。
 - **IEEE 754 结构**: 将 n 位划分为 $[s \mid e \mid f]$ 。指数部分采用偏置表示: $E = e + \text{Bias}$ ，其中 $\text{Bias} = 2^{k-1} - 1$ (k 为指数位宽)。
 - **特殊值**:
 - NaN: $e = 2^k - 1 \wedge f \neq 0$ 。
 - $\pm\infty$: $e = 2^k - 1 \wedge f = 0$ 。
 - ± 0 : $e = 0 \wedge f = 0$ 。
 - **浮点相等判定**: 引入容差 $\epsilon > 0$ ，定义等价关系 $a \equiv_{\epsilon} b \iff |a - b| < \epsilon$ 。

2.3 字符体系 (Character)

字符系统建立了从底层数字到人类符号的双射映射。

2.3.1 ANSI / ASCII

标准 ASCII 为映射 $\phi: \{0, \dots, 127\} \rightarrow \Sigma_{\text{ASCII}}$ ，物理存储占用 8 位 (1 字节)，最高位恒为 0。

2.3.2 Unicode 架构

1. **抽象字符集 (UCS)**: 定义全局字符空间 $\mathcal{U}$ ，每个字符 $c \in \mathcal{U}$ 对应唯一码点 $u \in \mathbb{N}$ ，记作 U+XXXX。
2. **编码方案 (UTF)**: 实现映射 $\psi: \mathcal{U} \rightarrow \mathbb{B}^*$ 。
 - **UTF-8**: 变长编码， $\psi(c) \in \mathbb{B}^8 \cup \mathbb{B}^{16} \cup \mathbb{B}^{24} \cup \mathbb{B}^{32}$ ，向下兼容 ASCII。
 - **UTF-16/32**: 定长/变长编码，常用于系统内核。

2.4 物理存储结构：线性表 (Linear List)

线性表是具有相同特性数据元素的有限序列 $L = \langle d_1, d_2, \dots, d_n \rangle$ ，其中 $d_i \in \mathcal{D}$ 。

2.4.1 顺序存储结构（顺序表 / 数组）

- 内存特征：分配连续物理地址空间 $[\beta, \beta + n \cdot \sigma)$ ，其中 β 为首地址， σ 为元素大小。
- 随机访问映射：定义索引函数 $\text{Addr} : \{0, \dots, n-1\} \rightarrow \text{Memory}$ ：

$$\text{Addr}(i) = \beta + i \cdot \sigma$$

- 多维压平（行优先）：对于 $M \times N$ 矩阵，逻辑坐标 (i, j) 映射为：

$$\text{Addr}(i, j) = \beta + (i \cdot N + j) \cdot \sigma$$

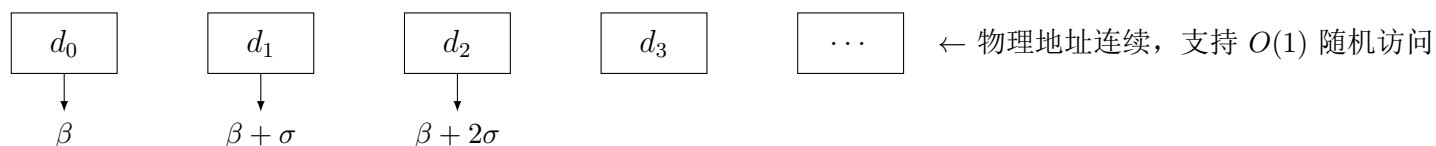


图 2.1: 一维顺序表物理内存布局

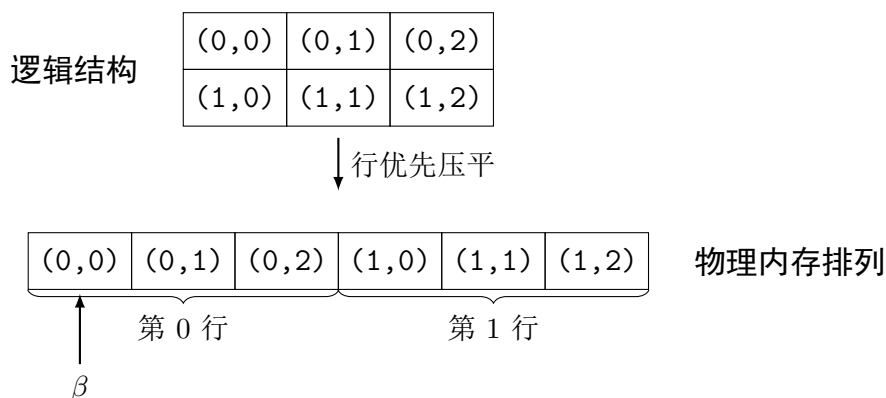


图 2.2: 二维数组行优先 (Row-Major) 压平排列

- 稀疏矩阵压缩：对于稀疏矩阵 $A \in \mathbb{R}^{M \times N}$ ，采用三元组集合表示：

$$\mathcal{T} = \{(i, j, A_{ij}) \mid A_{ij} \neq 0\}$$

2.4.2 链式存储结构（链表 / Link List）

- 内存特征：节点 $n_k = (d_k, p_k)$ 物理地址离散，其中 $d_k \in \mathcal{D}$ 为数据域， $p_k \in \text{Memory} \cup \{\emptyset\}$ 为指针域。
- 拓扑维系：链表 L 由节点序列构成，满足 $p_k = \text{Addr}(n_{k+1})$ ，且 $p_{\text{last}} = \emptyset$ 。
- 访问复杂度：丧失直接寻址能力，检索第 i 个元素需顺序遍历，时间复杂度 $O(n)$ 。

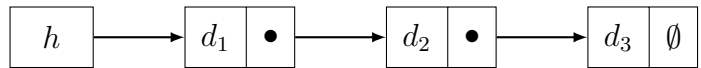


图 2.3: 单向链表 (Singly Linked List)

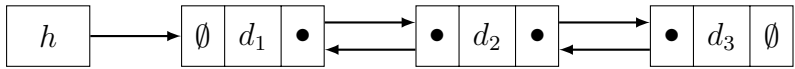


图 2.4: 双向链表 (Doubly Linked List)

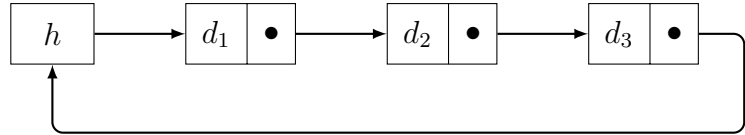


图 2.5: 循环链表 (Circular Linked List)

2.5 核心矛盾对比：顺序表 vs 链表

特性维度	顺序存储（数组）	链式存储（链表）
物理内存要求	连续区间 $[\beta, \beta + n\sigma)$	离散节点集合 $\{n_1, \dots, n_k\}$
元素访问效率	$O(1)$: $\text{Addr}(i) = \beta + i\sigma$	$O(n)$: 需顺序遍历指针链
插入与删除	$O(n)$: 需平移后续元素 $d_{i+1}, \dots, d_n$	$O(1)$: 仅需修改相邻节点指针域
空间利用率	预分配, 易产生空闲碎片; 扩容代价高	动态分配, 但指针域引入额外开销

第三章 Linear Structures and Abstract Data Types

3.1 栈 (Stack)

3.1.1 代数定义

栈 S 是定义在元素集 E 上的代数结构，由以下算子与公理刻画：

- 构造算子：

- $\text{empty} : \rightarrow S$ （空栈）
- $\text{push} : S \times E \rightarrow S$ （压入）

- 观测算子：

- $\text{pop} : S \rightarrow S \cup \{\perp\}$ （弹出）
- $\text{top} : S \rightarrow E \cup \{\perp\}$ （栈顶元素）
- $\text{isEmpty} : S \rightarrow \mathbb{B}$ （判空）

公理系统 ($\forall s \in S, e \in E$):

1. $\text{isEmpty}(\text{empty}) = \text{true}$
2. $\text{isEmpty}(\text{push}(s, e)) = \text{false}$
3. $\text{pop}(\text{push}(s, e)) = s$
4. $\text{top}(\text{push}(s, e)) = e$
5. $\text{pop}(\text{empty}) = \perp, \text{top}(\text{empty}) = \perp$

3.1.2 物理实现方案

1. **顺序栈**：映射到连续数组 $A[0..m-1]$ ，栈顶指针 $t \in \{-1, \dots, m-1\}$ 。push 操作需满足 $t < m-1$ ，否则栈溢出。
2. **链栈**：映射到单向链表，表头即为栈顶，无容量上限。

Algorithm 1 栈操作伪代码 (Stack Operations)

```
1: 数据结构: 数组  $A[0..m-1]$ , 整数  $t \leftarrow -1$ 
2: procedure PUSH( $e$ )
3:   if  $t = m - 1$  then
4:     return Error (Stack Overflow)
5:   end if
6:    $t \leftarrow t + 1$ 
7:    $A[t] \leftarrow e$ 
8: end procedure
9: procedure POP
10:  if  $t = -1$  then
11:    return Error (Stack Underflow)
12:  end if
13:   $e \leftarrow A[t]$ 
14:   $t \leftarrow t - 1$ 
15:  return  $e$ 
16: end procedure
```

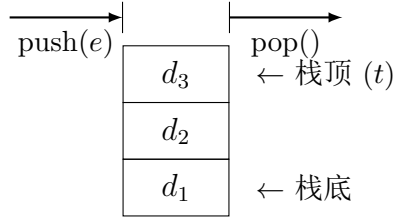


图 3.1: 栈 (Stack) 逻辑模型

3.2 队列 (Queue)

3.2.1 代数定义

队列 Q 是定义在元素集 E 上的代数结构:

- 构造算子: $\text{emptyQ} : \rightarrow Q$, $\text{enqueue} : Q \times E \rightarrow Q$
- 观测算子: $\text{dequeue} : Q \rightarrow Q \cup \{\perp\}$, $\text{front} : Q \rightarrow E \cup \{\perp\}$, $\text{isEmptyQ} : Q \rightarrow \mathbb{B}$

公理系统 ($\forall q \in Q, e \in E$):

1. $\text{isEmptyQ}(\text{emptyQ}) = \text{true}$
2. $\text{dequeue}(\text{enqueue}(\text{emptyQ}, e)) = \text{emptyQ}$
3. $\text{dequeue}(\text{enqueue}(\text{enqueue}(q, e_1), e_2)) = \text{enqueue}(\text{dequeue}(\text{enqueue}(q, e_1)), e_2)$
4. $\text{front}(\text{enqueue}(\text{emptyQ}, e)) = e$
5. 若 $\text{isEmptyQ}(q) = \text{false}$, 则 $\text{front}(\text{enqueue}(q, e)) = \text{front}(q)$

3.2.2 循环队列 (Circular Queue)

为消除假溢出，将数组索引映射到模 m 剩余类环 $\mathbb{Z}_m$ 。

- 指针移动: $f \leftarrow (f + 1) \bmod m$, $r \leftarrow (r + 1) \bmod m$ 。
- 队满判据: $(r + 1) \bmod m = f$ (牺牲一槽位)。
- 队空判据: $r = f$ 。

Algorithm 2 循环队列操作伪代码 (Circular Queue Operations)

```
1: 数据结构: 数组  $Q[0..m - 1]$ , 整数  $f \leftarrow 0, r \leftarrow 0$ 
2: procedure ENQUEUE( $e$ )
3:   if  $(r + 1) \bmod m = f$  then
4:     return Error (Queue Full)
5:   end if
6:    $Q[r] \leftarrow e$ 
7:    $r \leftarrow (r + 1) \bmod m$ 
8: end procedure
9: procedure DEQUEUE
10:  if  $r = f$  then
11:    return Error (Queue Empty)
12:  end if
13:   $e \leftarrow Q[f]$ 
14:   $f \leftarrow (f + 1) \bmod m$ 
15:  return  $e$ 
16: end procedure
```

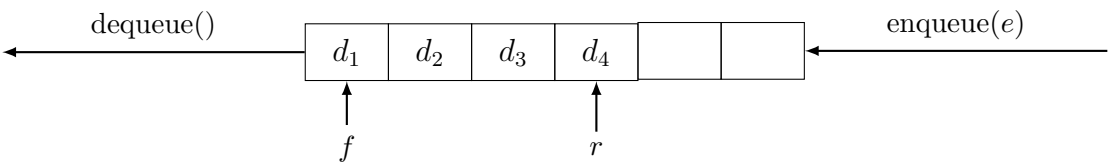


图 3.2: 普通队列 (Queue)

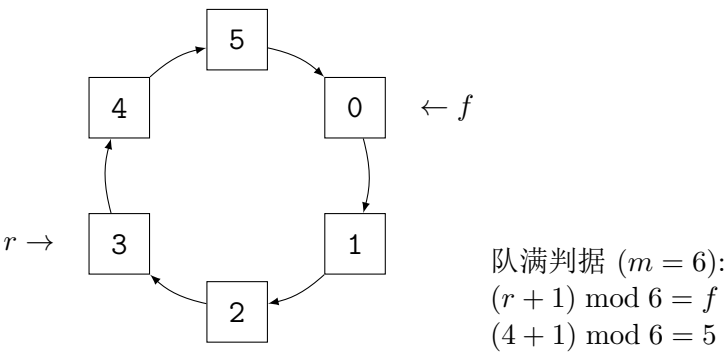


图 3.3: 循环队列 (Circular Queue)

3.3 字符串 (String)

3.3.1 形式化定义

- 字母表: Σ 为有限非空字符集合。
- 字符串集合: $\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k$, 其中 Σ^k 为长度为 k 的字符串集合。
- 空串: $\varepsilon \in \Sigma^0$ 。
- 连接运算: $\circ: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$, 满足结合律, 且 ε 为单位元。
- 物理存储: 在底层语言中, 字符串 $s \in \Sigma^*$ 映射为字符数组, 以终结符 $\# \notin \Sigma$ (如 C 语言的 $\backslash 0$) 界定边界。

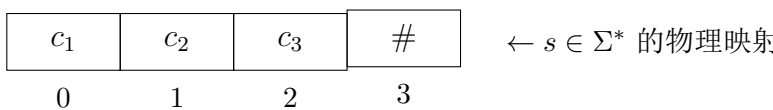


图 3.4: 字符串物理存储布局

3.4 高级映射与哈希表体系

3.4.1 字典 (Dictionary) / 映射 (Map)

- 逻辑定义: 映射关系 $\mathcal{M}: K \rightarrow V$, 其中 K 为键集, V 为值集, $\rightarrow$ 表示偏函数。
- 约束: $\forall k_1, k_2 \in \text{dom}(\mathcal{M}), \mathcal{M}(k_1) = \mathcal{M}(k_2) \implies k_1 = k_2$ (单射性)。

3.4.2 哈希表 (Hash Table)

通过哈希函数 $h: K \rightarrow \mathbb{Z}_m$ 将键空间压缩至有限数组槽位 $\{0, \dots, m-1\}$ 。

(1) 哈希函数

- 除留余数法: $h(k) = k \bmod m$, 通常取 m 为质数以最小化冲突。
- 理想哈希: $\forall k_i \neq k_j \implies h(k_i) \neq h(k_j)$ 。

3.4.3 哈希碰撞 (Hash Collision)

(1) 碰撞必然性

由鸽巢原理, 若 $|K| > m$, 则 $\exists k_i \neq k_j$ 使得 $h(k_i) = h(k_j)$ 。

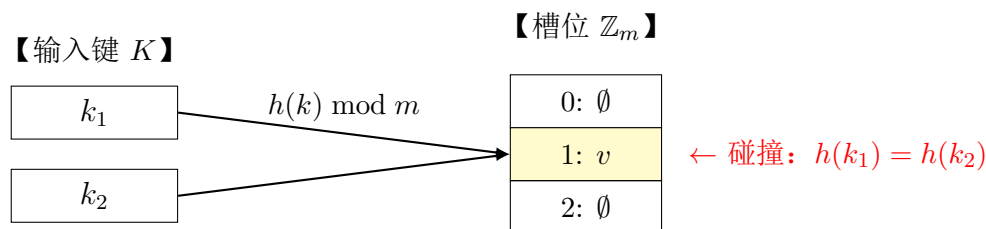


图 3.5: 哈希碰撞三大核心要素模型

(2) 冲突解决方案

1. 开放寻址法 (Open Addressing)

- 探测序列: $h_i(k) = (h(k) + \delta(i)) \bmod m$, 其中 $\delta(i)$ 为步长函数; 所有槽位都存放在主表数组中, 冲突时按序列继续探测。
- 线性探测 (Linear Probing): $\delta(i) = i$, 探测 $h(k), h(k) + 1, h(k) + 2, \dots$
 - 实现最简单、缓存友好, 期望探测次数 $O(1 + \alpha)$ 。
 - 缺点: 易产生主聚集 (Primary Clustering)——冲突键连续堆叠成块, 探测链越拖越长, 性能急剧恶化。
- 二次探测 (Quadratic Probing): $\delta(i) = i^2$, 探测 $h(k), h(k) + 1, h(k) + 4, h(k) + 9, \dots$
 - 探测间距随 i 增大而拉大, 同簇键迅速分散, 缓解主聚集 (次级聚集仍可能存在)。
 - 注意: $i^2 \bmod m$ 可能提前循环, 仅当 m 为质数且负载因子 $\alpha < 0.5$ 时才能保证遍历全部槽位。
- 双重散列 (Double Hashing): $h_i(k) = (h_1(k) + i \cdot h_2(k)) \bmod m$, 其中 $h_2(k) \not\equiv 0 \pmod m$
 - 典型取法: $h_1(k) = k \bmod m$, $h_2(k) = 1 + (k \bmod (m - 1))$, 步长随键变化且与 m 互质 (m 与 $m - 1$ 恒互质)。
 - 每个键拥有独立的探测序列, 分布最均匀, 最接近理想随机探测, 冲突性能最优。
- 开放寻址公共缺陷: 删除不能直接置空 (会截断后续键的探测链), 须以「墓碑」标记被删槽位; 负载因子过高时须再散列 (Rehashing) 扩容重建。

2. 链地址法 (Chaining)

- 每个槽位 $i \in \mathbb{Z}_m$ 维护一个链表 $L_i = \{(k, v) \mid h(k) = i\}$ 。
- 检索时间复杂度: 期望 $O(1 + \alpha)$, 其中 $\alpha = n/m$ 为负载因子。
- 动态重构: 当 $|L_i| > \tau$ (阈值) 时, 将链表重构为自平衡二叉搜索树 (如红黑树), 最坏检索复杂度降为 $O(\log n)$ 。

Algorithm 3 哈希表查找与插入伪代码 (Hash Table Operations)

1: 数据结构: 数组 $T[0..m-1]$ (初始为 $\emptyset$), 哈希函数 $h(k)$
2: **procedure** SEARCH(k)
3: $i \leftarrow 0$
4: **repeat**
5: $j \leftarrow (h(k) + i) \bmod m$
6: **if** $T[j] = \emptyset$ **or** $T[j].key = k$ **then**
7: **return** $T[j]$
8: **end if**
9: $i \leftarrow i + 1$
10: **until** $i = m$
11: **return** Error (Not Found)
12: **end procedure**
13: **procedure** INSERT(k, v)
14: $item \leftarrow$ Search(k)
15: **if** $item \neq$ Error **then**
16: $item.value \leftarrow v$
17: **else**
18: $i \leftarrow 0$
19: **while** True **do**
20: $j \leftarrow (h(k) + i) \bmod m$
21: **if** $T[j] = \emptyset$ **then**
22: $T[j] \leftarrow \langle k, v \rangle$
23: **return**
24: **end if**
25: $i \leftarrow i + 1$
26: **end while**
27: **end if**
28: **end procedure**

▷ 线性探测

▷ 更新

槽位 i

0	→	(k_1, v_1)	→	(k_2, v_2)	→	$\emptyset$
1	→	$\emptyset$				
2	→	红黑树				

(冲突数 $> \tau$ 时重构,
检索复杂度 $O(\log n)$)

图 3.6: 链地址法与动态重构模型

第四章 Non-linear Structures I: Trees and Binary Trees

树是一种一对多的非线性层次逻辑结构。在图论中，树被严密定义为：任意两个顶点间有且仅有一条路径的无环连通图。

4.1 通用树的基础属性

- 形式化定义：树 $T = (V, E)$ ，其中 V 为有限顶点集， $E \subseteq V \times V$ 为边集，满足 $|E| = |V| - 1$ 且连通。
- 节点分类：
 - 根节点 (Root)：唯一入度为 0 的节点 $r \in V$ 。
 - 叶子节点 (Leaf)：出度为 0 的节点集合 $L = \{v \in V \mid \text{out-degree}(v) = 0\}$ 。
 - 分支节点 (Internal)： $I = V \setminus (L \cup \{r\})$ 。
- 形态指标：
 - 度数 (Degree)： $\deg(v) = |\{u \mid (v, u) \in E\}|$ 。树的度 $D = \max_{v \in V} \deg(v)$ 。
 - 高度 (Height)： $H(T) = \max_{v \in V} \text{depth}(v)$ 。

4.2 二叉树 (Binary Tree) 及其形态特征

- 递归定义：二叉树 B 要么是空集 $\emptyset$ ，要么是一个三元组 $\langle d, L, R \rangle$ ，其中 d 为根节点数据， L 和 R 为互不相交的二叉树（左子树与右子树）。
- 节点约束： $\forall v \in V, \deg(v) \in \{0, 1, 2\}$ ，且严格区分左右子树。

根据形态与平衡度，二叉树有以下经典状态：

1. 偏斜树 (Skewed Tree)： $\forall v, \deg(v) \leq 1$ 。退化为线性结构，检索复杂度 $O(n)$ 。
2. 完全二叉树 (Complete Tree)：节点按层序编号 $1 \dots n$ ，与满二叉树前 n 个节点一一对应。
3. 满二叉树 (Full Tree)： $\forall v, \deg(v) \in \{0, 2\}$ ，且所有叶子节点深度相同。
4. 二叉搜索树 (BST)：满足偏序关系 $\forall x \in L, x.val < d.val \wedge \forall y \in R, y.val > d.val$ 。

5. 平衡二叉树 (AVL): $\forall v \in V, |H(L_v) - H(R_v)| \leq 1$ 。

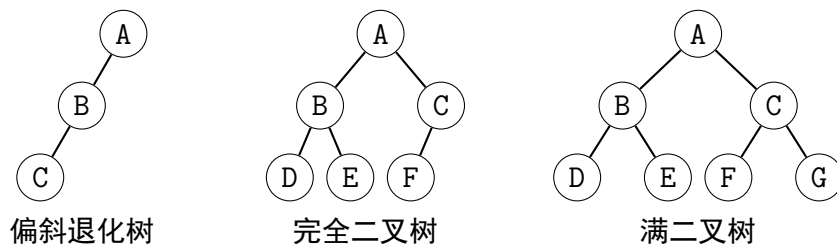


图 4.1: 二叉树核心形态对比

4.3 树的物理存储

1. 顺序存储法（完全二叉树）

- 映射到数组 $A[0..n-1]$ 。
- 隐式纽带: $\forall i \in \{0, \dots, n-1\}$,
 - 左子节点索引: $l(i) = 2i + 1$ (若 $< n$)
 - 右子节点索引: $r(i) = 2i + 2$ (若 $< n$)
 - 父节点索引: $p(i) = \lfloor (i-1)/2 \rfloor$ (若 $i > 0$)

2. 链式存储法（二叉链表）

- 节点结构: $n = \langle lc, d, rc \rangle$, 其中 $lc, rc \in \text{Node} \cup \{\emptyset\}$ 。
- 通用树转换（孩子兄弟表示法）: $n = \langle \text{firstChild}, d, \text{nextSibling} \rangle$ 。

4.4 树的遍历 (Tree Traversal)

将非线性结构 T 映射为线性序列 $\sigma \in V^*$ 。

4.4.1 递归遍历 (Recursive Traversal)

Algorithm 4 二叉树递归遍历伪代码 (Binary Tree Traversal)

1: procedure PREORDER(u)	▷ 前序遍历
2: if $u = \emptyset$ then	
3: return	
4: end if	
5: Visit (u)	
6: Preorder(u .left)	
7: Preorder(u .right)	
8: end procedure	
9: procedure INORDER(u)	▷ 中序遍历
10: if $u = \emptyset$ then	
11: return	
12: end if	
13: Inorder(u .left)	
14: Visit (u)	
15: Inorder(u .right)	
16: end procedure	
17: procedure POSTORDER(u)	▷ 后序遍历
18: if $u = \emptyset$ then	
19: return	
20: end if	
21: Postorder(u .left)	
22: Postorder(u .right)	
23: Visit (u)	
24: end procedure	

4.4.2 非递归遍历 (Iterative Traversal)

利用显式栈 S 模拟系统调用栈，消除递归开销。

Algorithm 5 中序遍历迭代伪代码 (Iterative Inorder)

1: procedure INORDERITERATIVE($root$)	
2: $S \leftarrow \emptyset$	▷ 初始化空栈
3: $curr \leftarrow root$	
4: while $curr \neq \emptyset$ or $S \neq \emptyset$ do	
5: while $curr \neq \emptyset$ do	
6: Push($S, curr$)	
7: $curr \leftarrow curr$.left	
8: end while	
9: $curr \leftarrow$ Pop(S)	
10: Visit ($curr$)	
11: $curr \leftarrow curr$.right	
12: end while	
13: end procedure	

4.4.3 线索二叉树 (Threaded Binary Tree)

为消除遍历时的栈空间开销 $O(H)$ ，利用空指针域存储遍历序中的前驱与后继。

- 数学定义：对于节点 u ，定义标记位 $LTag, RTag \in \{0, 1\}$ ：
 - 若 $u.left = \emptyset$ ，则 $u.left \leftarrow pre(u)$ （中序前驱），且 $LTag(u) = 1$ 。
 - 若 $u.right = \emptyset$ ，则 $u.right \leftarrow succ(u)$ （中序后继），且 $RTag(u) = 1$ 。
- 性质：线索化后，遍历操作退化为线性扫描，空间复杂度降至 $O(1)$ 。

4.4.4 遍历序列与树的唯一性

1. 唯一确定性定理：

- 前序 + 中序 $\implies$ 唯一确定二叉树。
- 后序 + 中序 $\implies$ 唯一确定二叉树。
- 前序 + 后序 $\implies$ 仅当树为满二叉树时唯一确定，否则存在歧义（无法区分左右子树边界）。

2. 构造算法：

Algorithm 6 由前序与中序序列构造二叉树

```

1: 输入: 前序序列  $P$ , 中序序列  $I$ 
2: procedure BUILDTREE( $P, I$ )
3:   if  $P = \emptyset$  or  $I = \emptyset$  then
4:     return  $\emptyset$ 
5:   end if
6:    $root \leftarrow P[0]$  ▷ 前序首元素为根
7:    $k \leftarrow \text{Index}(I, root)$  ▷ 在中序中定位根
8:    $I_L \leftarrow I[0..k-1], I_R \leftarrow I[k+1..|I|-1]$ 
9:    $P_L \leftarrow P[1..|I_L|], P_R \leftarrow P[|I_L|+1..|P|-1]$ 
10:   $root.left \leftarrow \text{BuildTree}(P_L, I_L)$ 
11:   $root.right \leftarrow \text{BuildTree}(P_R, I_R)$ 
12:  return  $root$ 
13: end procedure

```

4.5 堆体系 (Heap)

- 定义：完全二叉树 T 满足堆序性质。
- 大顶堆 (Max Heap): $\forall v \in V, v.val \geq \text{leftChild}(v).val \wedge v.val \geq \text{rightChild}(v).val$ 。
- 小顶堆 (Min Heap): $\forall v \in V, v.val \leq \text{leftChild}(v).val \wedge v.val \leq \text{rightChild}(v).val$ 。
- 功能：提供 $O(1)$ 的最值访问与 $O(\log n)$ 的动态插入/删除，是优先队列的物理实现。

Algorithm 7 堆下滤操作伪代码 (Sift-Down for Max Heap)

```
1: 输入: 数组  $A$ , 节点索引  $i$ , 堆大小  $n$ 
2: procedure SIFTDOWN( $i, n$ )
3:    $largest \leftarrow i$ 
4:    $l \leftarrow 2i + 1$                                 ▷ 左孩子
5:    $r \leftarrow 2i + 2$                                 ▷ 右孩子
6:   if  $l < n$  and  $A[l] > A[largest]$  then
7:      $largest \leftarrow l$ 
8:   end if
9:   if  $r < n$  and  $A[r] > A[largest]$  then
10:     $largest \leftarrow r$ 
11:  end if
12:  if  $largest \neq i$  then
13:    Swap( $A[i], A[largest]$ )
14:    SiftDown( $largest, n$ )                                ▷ 递归下滤
15:  end if
16: end procedure
```

第五章 Non-linear Structures II: Graph Theory

图是一种多对多的网状逻辑结构，用于建模现实世界中复杂的关联网络。

5.1 图的形式化定义

图 G 定义为二元组 $G = (V, E)$:

- $V = \{v_1, v_2, \dots, v_n\}$ 为有限顶点集合。
- $E \subseteq V \times V$ 为边集合。
- 若 E 为无序对集合 $\{\{u, v\} \mid u, v \in V\}$ ，则为无向图；若为有序对集合 $\{(u, v) \mid u, v \in V\}$ ，则为有向图。

扩展属性:

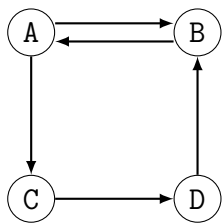
- 权重函数: $w : E \rightarrow \mathbb{R}$ ，赋予每条边实数值。
- 邻接映射: $N : V \rightarrow \mathcal{P}(V)$ ，其中 $N(u) = \{v \mid (u, v) \in E\}$ 。

5.2 边的拓扑分类

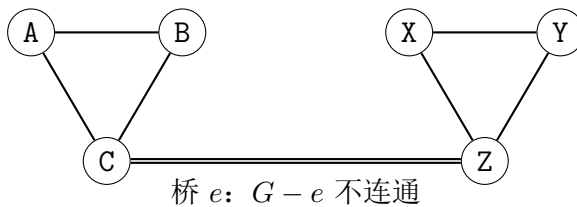
1. 有向无环图 (DAG): 有向图且不存在序列 $\langle v_1, v_2, \dots, v_k \rangle$ 使得 $(v_i, v_{i+1}) \in E$ 且 $v_1 = v_k$ 。
2. 连通图 (Connected Graph): 无向图中 $\forall u, v \in V$ ，存在路径 $P = \langle u = x_0, x_1, \dots, x_k = v \rangle$ 使得 $(x_i, x_{i+1}) \in E$ 。

5.3 连通性深度分析

- 强连通 (Strongly Connected): 有向图中 $\forall u, v \in V$ ，存在 $u \rightarrow v$ 路径且存在 $v \rightarrow u$ 路径。
- 弱连通 (Weakly Connected): 忽略边方向后形成的无向图是连通的。
- 桥 (Bridge): 边 $e \in E$ 为桥 $\iff$ 移除 e 后图的连通分量数增加，即 $G' = (V, E \setminus \{e\})$ 不连通。



强连通有向图



包含“桥”的断连风险图

图 5.1: 图的位置连通性形态与“桥”

5.4 图的降维与拓扑排序

Algorithm 8 图遍历伪代码 (Graph Traversal)

```

1: 全局变量: Visited  $\subseteq V$  (初始为  $\emptyset$ )
2: procedure BFS( $s$ ) ▷ 广度优先搜索
3:    $Q \leftarrow \{s\}$ 
4:   Visited  $\leftarrow$  Visited  $\cup \{s\}$ 
5:   while  $Q \neq \emptyset$  do
6:      $u \leftarrow$  Dequeue( $Q$ )
7:     Visit( $u$ )
8:     for all  $v \in N(u) \setminus \text{Visited}$  do
9:       Visited  $\leftarrow$  Visited  $\cup \{v\}$ 
10:      Enqueue( $Q, v$ )
11:    end for
12:  end while
13: end procedure
14: procedure DFS( $u$ ) ▷ 深度优先搜索
15:   Visited  $\leftarrow$  Visited  $\cup \{u\}$ 
16:   Visit( $u$ )
17:   for all  $v \in N(u) \setminus \text{Visited}$  do
18:     DFS( $v$ )
19:   end for
20: end procedure

```

• 拓扑排序 (Topological Sort):

- 对象: DAG。
- 目标: 构造线性序列 $\sigma = \langle v_1, \dots, v_n \rangle$ 使得 $\forall (u, v) \in E$, 若 $u = v_i, v = v_j$, 则 $i < j$ 。

5.5 图的物理存储实现

1. 邻接矩阵 (Adjacency Matrix)

- 定义: $A \in \{0, 1\}^{n \times n}$, 其中 $A_{ij} = 1 \iff (v_i, v_j) \in E$ 。
- 带权图: $A_{ij} = w(v_i, v_j)$ 或 ∞ 。
- 空间复杂度: $O(|V|^2)$, 适合稠密图。

2. 邻接表 (Adjacency List)

- 定义：数组 $\text{Adj}[1..n]$ ，其中 $\text{Adj}[i] = \{v \mid (v_i, v) \in E\}$ 以链表形式存储。
- 空间复杂度： $O(|V| + |E|)$ ，适合稀疏图。

第六章 Algorithm Definition and Asymptotic Analysis

本章探讨评价算法优劣的统一数学度量衡，建立时间与空间复杂度的渐进趋势评估模型。

6.1 算法的定义与四大核心特征

算法 (Algorithm) 是指解决特定问题的一组明确的、有穷的、可执行的指令序列。一个合格的算法必须具备以下四大核心特征：

1. **有穷性 (Finiteness)**: 算法必须在执行有限个操作步骤后自动结束，不能陷入死循环，且每一步都应在有穷时间内完成。
2. **确定性 (Definiteness)**: 算法中的指令每一步都必须有确切的含义，在任何条件下都具有唯一一条执行路径，不产生任何歧义。
3. **可行性 (Effectiveness)**: 算法中涉及的所有操作均是最基础、可执行的，能够通过已经实现的底层基本运算在有限时间内准确运行。
4. **输入/输出 (Input / Output)**: 一个算法拥有零个或多个输入 (作为算法处理的初始物理量)，并必然产生一个或多个输出 (作为算法执行的结果回馈)。

6.2 算法的描述媒介

根据“信息流动方向”与“面向对象 (人或机器)”的不同，算法的描述主要分为两大阵营、四种流派：

- 面向人类思维 (人写人看):

1. **自然语言**: 用人类日常语言描述。优点是通俗易懂，缺点是过于冗长且容易产生语义歧义。
2. **流程图 (Flowchart)**: 利用图形化、具有严密逻辑关系的有向图来表达控制流。
3. **伪代码 (Pseudo-code)**: 介于自然语言与编程语言之间。直接利用公式与控制流算子进行映射，具备接近代码的严密性，是学术界描述算法的标准媒介。

- 面向底层执行 (人写机看):

4. **高级编程语言**: 如 C++、Java、Python 等。直接翻译为计算机硬件可以理解并编译运行的二进制指令。

6.3 算法的时间复杂度 (Time Complexity)

评价一个算法的优劣不能依靠绝对的计时。科学的评估方法是分析输入规模 n 与资源消耗规模之间的渐进趋势关系。

6.3.1 大 O 表示法的渐进上界定义

大 O 符号用于寻求算法执行时间随着规模 n 放大时的渐进上界。其严格数学定义为：若存在正连续常数 c 和 n_0 ，使得当 $n \geq n_0$ 时，算法的实际运行时间 $T(n)$ 均满足：

$$T(n) \leq c \cdot f(n)$$

则记作 $T(n) = O(f(n))$ 。它忽略了低阶项与常数系数，专注于长远增长趋势。

6.3.2 复杂度阶数

随着输入规模 n 的爆炸式增长，不同复杂度阶数的资源消耗会产生明显的趋势分流。

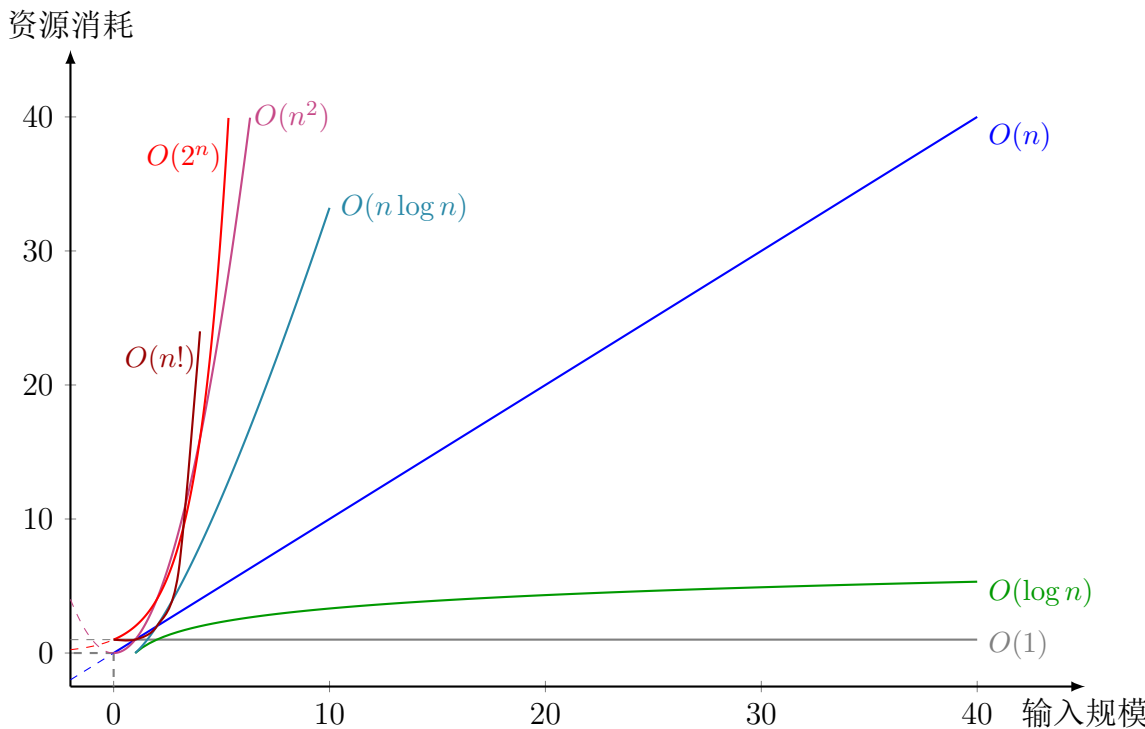


图 6.1: 大 O 渐进曲线趋势对比

6.3.3 复杂度组合乘法原理

当算法的各个模块相互嵌套或串联时，其大 O 符号满足乘法运算法则：

$$O(a) \cdot O(b) = O(a \cdot b)$$

6.4 算法的空间复杂度 (Space Complexity)

空间复杂度用于衡量随着输入规模 n 的扩大，算法运行期间额外占用的辅助存储空间的增幅趋势。

6.4.1 $O(1)$ 常数级辅助空间 (In-place 原地置换)

算法在原输入数据的存储空间上直接进行修改与操作。整个过程不需要开辟随 n 增长的新空间，仅需要常数个辅助变量用于临时中转。

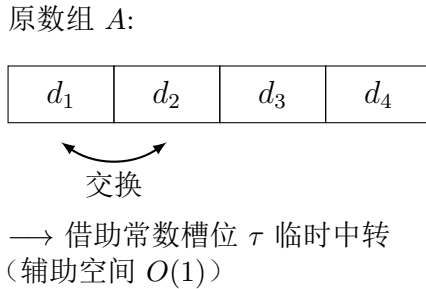


图 6.2: 原地置换 (Swap In-place) 内存状态机模型

6.4.2 $O(n)$ 线性级辅助空间 (Clone 副本生成)

在算法执行期间（如为了保持原数据不被破坏，或者为了进行高效率的哈希映射），必须在物理内存中动态生成一份与原输入规模等大的副本 (Clone)。其开辟的内存槽位随 n 同步线性增长。

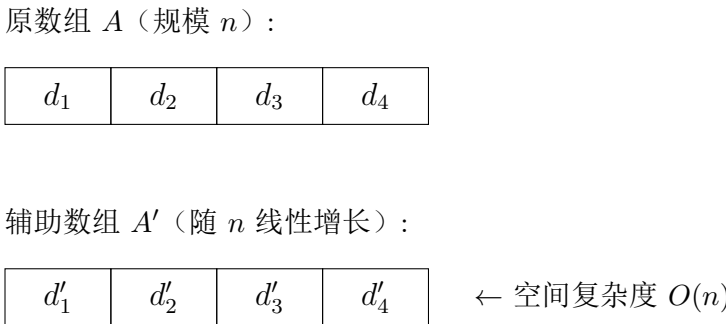


图 6.3: 副本克隆 (Clone) 内存空间扩张模型

第七章 Dynamic Classic Algorithms and Core Operators

本章探讨从静态结构向动态算子的演进，涵盖算法设计范式、排序、查找以及高级图论算法。

7.1 顶层设计范式

7.1.1 分治策略 (Divide and Conquer)

将规模 n 的问题 P 划分为 a 个子问题 $\{P_1, \dots, P_a\}$ ，每个规模为 n/b 。时间递推关系：

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

由主定理 (Master Theorem) 决定渐进复杂度。

7.1.2 贪心算法 (Greedy)

每一步选择局部最优解 $s_i = \arg \max_{x \in C} \mathcal{F}(x)$ 。全局最优需满足：

1. 最优子结构： P 的最优解包含子问题 P' 的最优解。
2. 贪心选择性： 可通过局部最优序列构造全局最优解。

7.2 排序算法算子 (Sort)

输入序列 $A = \langle a_1, \dots, a_n \rangle$, 目标输出 $A' = \langle a'_1, \dots, a'_n \rangle$ 使得 $a'_1 \leq a'_2 \leq \dots \leq a'_n$ 。

7.2.1 比较类排序

1. 冒泡排序: 通过相邻交换减少逆序对数 $I(A) = |\{(i, j) \mid i < j \wedge a_i > a_j\}|$ 。每轮将最大值移至末尾。
2. 选择排序: 第 k 轮选择 $a'_k = \min\{a_k, \dots, a_n\}$ 并与 a_k 交换。比较次数固定为 $\frac{n(n-1)}{2}$ 。
3. 插入排序: 维护已排序前缀 $A[1..k]$, 将 a_{k+1} 插入正确位置。最好情况 $O(n)$ (已有序)。
4. 快速排序: 选取基准 p , 划分 A 为 $A_L = \{x \in A \mid x \leq p\}$ 与 $A_R = \{x \in A \mid x > p\}$ 。递归排序 A_L, A_R 。
5. 归并排序: $T(n) = 2T(n/2) + O(n)$ 。将 A 对半分割后合并, 稳定且 $O(n \log n)$, 但需 $O(n)$ 辅助空间。
6. 堆排序: 建堆 $O(n)$, 重复提取根节点并下滤。时间 $O(n \log n)$, 空间 $O(1)$ 。

Algorithm 9 快速排序伪代码 (Quick Sort)

```
1: procedure QUICKSORT( $A, l, r$ )
2:   if  $l \geq r$  then
3:     return
4:   end if
5:    $p \leftarrow \text{Partition}(A, l, r)$  ▷ 划分操作
6:   QuickSort( $A, l, p - 1$ )
7:   QuickSort( $A, p + 1, r$ )
8: end procedure
9: procedure PARTITION( $A, l, r$ )
10:   $pivot \leftarrow A[r]$  ▷ 选取末尾元素为基准
11:   $i \leftarrow l - 1$ 
12:  for  $j \leftarrow l$  to  $r - 1$  do
13:    if  $A[j] \leq pivot$  then
14:       $i \leftarrow i + 1$ 
15:      Swap( $A[i], A[j]$ )
16:    end if
17:  end for
18:  Swap( $A[i + 1], A[r]$ )
19:  return  $i + 1$ 
20: end procedure
```

7.2.2 非比较类排序

1. 计数排序: 统计频次 $C[x] = |\{a_i \mid a_i = x\}|$, 计算前缀和 $S[x] = \sum_{k \leq x} C[k]$ 确定位置。
2. 桶排序: 将值域 $[Min, Max]$ 划分为 k 个区间 $B_1, \dots, B_k$, 桶内排序后串联。
3. 基数排序: 按位 $d = 0 \dots D - 1$ 进行稳定排序 (通常用计数排序)。

7.3 查找与并查集

7.3.1 二分查找 (Binary Search)

作用于有序数组 A 。维护搜索区间 $[l, r]$ ，取中点 $m = \lfloor (l + r) / 2 \rfloor$ 。

若 $A[m] = x \implies$ 找到; 若 $A[m] < x \implies l = m + 1$; 否则 $r = m - 1$

时间复杂度 $O(\log n)$ 。

7.3.2 并查集 (Union-Find)

维护不相交集合族 $\mathcal{S} = \{S_1, \dots, S_k\}$, $S_i \subseteq V$ 。

- $\text{Find}(x)$: 返回 x 所在集合的代表元。
- $\text{Union}(x, y)$: 合并 S_x 与 S_y 。

引入按秩合并与路径压缩后，摊还复杂度 $O(\alpha(n))$ ，其中 α 为反阿克曼函数。

7.4 图论核心算法

7.4.1 基础遍历

维护已访问集合 $\text{Visited} \subseteq V$ 。

BFS

1. 初始化 $Q = \{s\}$, $\text{Visited} = \{s\}$ 。
2. 当 $Q \neq \emptyset$, 弹出 u 。
3. $\forall v \in N(u) \setminus \text{Visited}$: $\text{Visited} \leftarrow \text{Visited} \cup \{v\}$, $Q \leftarrow Q \cup \{v\}$ 。

DFS

递归定义: $\text{DFS}(u) \implies \text{Visited} \leftarrow \text{Visited} \cup \{u\} \wedge \forall v \in N(u) \setminus \text{Visited}, \text{DFS}(v)$ 。

7.4.2 最短路径算法

维护距离函数 $D: V \rightarrow \mathbb{R} \cup \{\infty\}$ 。松弛操作:

$$\text{若 } D[u] + w(u, v) < D[v] \implies D[v] \leftarrow D[u] + w(u, v)$$

1. **Dijkstra**: 贪心策略。维护集合 S (已确定最短路径), 每次选 $u \in V \setminus S$ 使 $D[u]$ 最小, 加入 S 并松弛邻边。不支持负权边。
2. **Bellman-Ford**: 对 E 进行 $|V| - 1$ 轮全量松弛。若第 $|V|$ 轮仍可松弛, 则存在负环。
3. **Floyd-Warshall**: 动态规划。 $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$ 。支持负权, 但负环导致 $D[i][i] < 0$ 。

Algorithm 10 Dijkstra 最短路径伪代码

```
1: 输入: 图  $G = (V, E)$ , 源点  $s$ , 权重  $w$ 
2: 初始化:  $\forall v \in V, D[v] \leftarrow \infty, D[s] \leftarrow 0$ 
3:  $S \leftarrow \emptyset$  ▷ 已确定最短路径集合
4:  $Q \leftarrow V$  ▷ 优先队列
5: while  $Q \neq \emptyset$  do
6:    $u \leftarrow \text{ExtractMin}(Q)$  ▷ 取出  $D[u]$  最小的节点
7:    $S \leftarrow S \cup \{u\}$ 
8:   for all  $v \in N(u)$  do
9:     if  $D[u] + w(u, v) < D[v]$  then ▷ 松弛操作
10:       $D[v] \leftarrow D[u] + w(u, v)$ 
11:       $\text{DecreaseKey}(Q, v, D[v])$ 
12:     end if
13:   end for
14: end while
```

7.4.3 最小生成树 (MST)

1. **Prim:** 从任意 $r \in V$ 开始, 维护树节点集 T 。每次选横切边 (u, v) 中 $w(u, v)$ 最小且 $u \in T, v \notin T$ 的边, 将 v 加入 T 。
2. **Kruskal:** 将 E 按权重升序排列。依序选边 (u, v) , 若 u, v 不在同一连通分量 (用并查集判定), 则加入 MST。

7.4.4 拓扑排序: Kahn 算法

1. 计算所有顶点入度 $\text{indeg}(v) = |\{u \mid (u, v) \in E\}|$ 。
2. 初始化队列 $Q = \{v \mid \text{indeg}(v) = 0\}$ 。
3. 弹出 u , 加入序列 σ 。 $\forall v \in N(u), \text{indeg}(v) \leftarrow \text{indeg}(v) - 1$; 若为 0 则入队。
4. 若 $|\sigma| < |V|$, 则图含环。

7.5 高级工程专项算子

7.5.1 网络流 (Network Flow)

图 $G = (V, E)$ 带容量 $c: E \rightarrow \mathbb{R}^+$ 。流 $f: E \rightarrow \mathbb{R}$ 满足:

1. 容量限制: $0 \leq f(u, v) \leq c(u, v)$
2. 流量守恒: $\forall u \in V \setminus \{s, t\}, \sum_v f(u, v) = 0$

Edmonds-Karp: 在残量网络 $c_f(u, v) = c(u, v) - f(u, v)$ 中用 BFS 找增广路, 更新 f 直至无增广路。复杂度 $O(|V| \cdot |E|^2)$ 。

7.5.2 二分图最大匹配

二分图 $G = (X \cup Y, E)$ 。匹配 $M \subseteq E$ 满足任意两条边无公共顶点。**匈牙利算法:** 寻找增广路 P (交替未匹配/匹配边, 端点均未匹配)。若存在, 则 $M' = M \oplus P$ (对称差) 为更大匹配。重复至无增广路。

7.5.3 强连通分量 (SCC): Tarjan 算法

基于 DFS 维护时间戳 $DFN[u]$ 与追溯值 $LOW[u]$ 。

- 初始化: $DFN[u] = LOW[u] = \text{timer}$, u 入栈。
- 遍历邻接点 v :
 - 若 v 未访问: $DFS(v)$, $LOW[u] = \min(LOW[u], LOW[v])$
 - 若 v 在栈中: $LOW[u] = \min(LOW[u], DFN[v])$
- 若 $LOW[u] = DFN[u]$, 则栈中 u 及上方节点构成一个 SCC, 批量弹栈。

时间复杂度 $O(|V| + |E|)$ 。

第八章 Algorithm Comparison

8.1 核心排序算法算子对比

算法	最好	最坏	平均	空间	稳定性	核心特征
冒泡	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	消除逆序对, 对有序敏感
选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	比较次数固定 $\frac{n(n-1)}{2}$
插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	顺移偏置, 适合近似有序
快速	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$	不稳定	分治划分, 基准随机化防退化
归并	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定	严格分治合并, 需辅助数组
堆排	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定	完全二叉树隐式维护偏序
计数	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	稳定	前缀和寻址, 突破比较下界

表 8.1: 核心排序算法算子对比

8.2 高级图论与检索算子对比

算法	目标	时间	空间	核心依赖	限制/判据
二分查找	有序检索	$O(\log n)$	$O(1)$	顺序存储对切	需连续且绝对有序
并查集	集合维护	$O(\alpha(n))$	$O(n)$	树森林、路径压缩	摊还接近 $O(1)$
Dijkstra	单源最短路	$O(E \log V)$	$O(V)$	优先队列、贪心	不支持负权边
Bellman-Ford	单源带权	$O(V \cdot E)$	$O(V)$	$ V - 1$ 轮松弛	第 $ V $ 轮松弛成功 $\implies$ 负环
Floyd-Warshall	多源最短路	$O(V ^3)$	$O(V ^2)$	状态转移矩阵	支持负权, 负环 $\implies D[i][i] < 0$
Prim	MST	$O(E \log V)$	$O(V)$	割定理、横切边	适合稠密图
Kruskal	MST	$O(E \log E)$	$O(E)$	边升序、并查集	适合稀疏图
Kahn	拓扑序	$O(V + E)$	$O(V)$	入度统计、队列	$ \sigma < V \implies$ 有环
Tarjan	SCC	$O(V + E)$	$O(V)$	DFS 双时间戳、栈	$LOW[u] = DFN[u] \implies$ 弹栈

表 8.2: 高级图论与检索算子对比